

Automatisation Web

Avec Playwright

<https://github.com/imhah-hassan/playwright-ts-training>

→ 20 mars 2026

Logistique



Assiduité et émargement



Pauses



Interactivité et questions



Utilisation de laptops et smartphones



Evacuation

Sommaire

- 0. Introduction aux tests Web
- 1. Installation et configuration Playwright
- 2. Identification des objets Web
- 3. Les assertions
- 4. Utilisation des données CSV
- 5. Cas pratique

0. Introduction aux tests Web



Pourquoi automatiser les tests ?

- Gagner du temps (sur les campagnes de TNR)
- Permettre aux développeurs de se focaliser sur le développement
- Réduire le nombre d'anomalies / Eviter les régressions / Améliorer la qualité du produit

Une application informatique est développée et peut ensuite être maintenue pendant plusieurs années

Tout au long de son existence, elle va subir des modifications (corrections, nouvelles fonctionnalités, migrations techniques...)

Chaque livraison de version va nécessiter des tests pour vérifier les nouveautés, mais aussi garantir le fonctionnement des fonctions existantes (non régression)

Dans un cycle agile, les nouvelles versions sont livrées aux testeurs sur un rythme très fréquent, à chaque itération (1-2 semaines), voire plusieurs fois au sein d'une même itération

Pourquoi automatiser les tests ?

La fréquence des livraisons de versions s'est considérablement accélérée

Les applications développées évoluent plus rapidement en fonction des changements plus fréquents exigés par le business

Les tests doivent donc être effectués plus souvent et plus vite

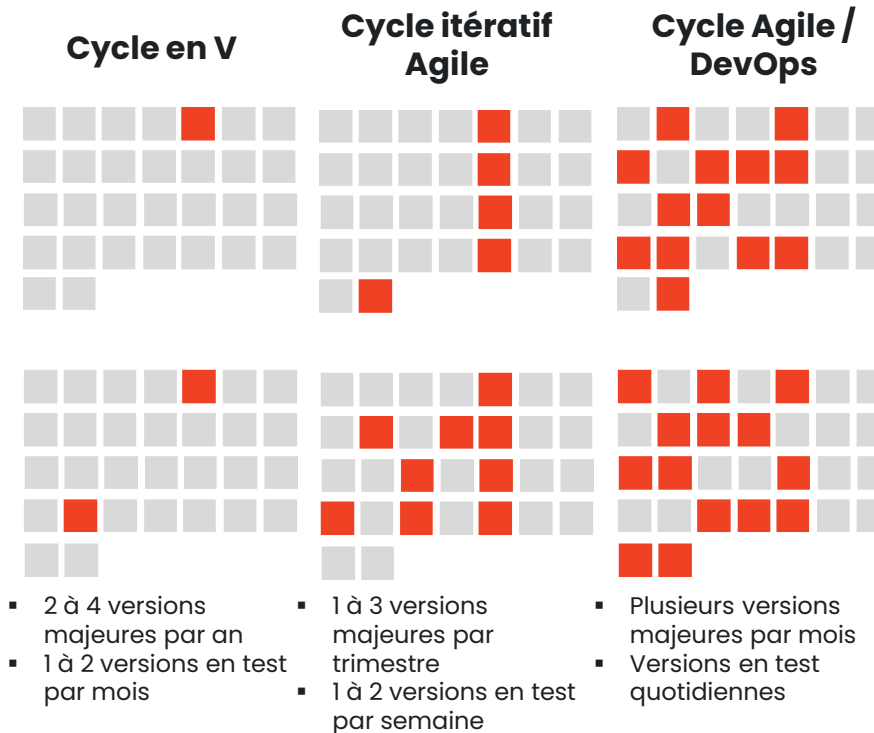
Ils sont effectués à chaque itération sur des versions en cours de construction

Ils ne doivent cependant pas compromettre la qualité

L'exécution fréquente des tests de non-régression devient inévitable et nécessite leur automatisation. Le reporting de test doit restituer la qualité d'ensemble

La qualité devient l'affaire de tous

L'ensemble de l'équipe doit participer à l'effort de test



Avantages et inconvénients

Tester plus vite

- Réduire les charges de test manuel
- Réduire le travail répétitif
- Augmenter la productivité des tests

Tester plus

- Elargir le périmètre des tests
- Concentrer les tests manuels sur les tâches à valeur ajoutée (nouvelles fonctionnalités, tests exploratoires...)

Tester mieux

- Diminuer les erreurs de manipulation
- Fournir une évaluation objective
- Capitaliser d'une campagne à l'autre
- Améliorer l'accessibilité des informations

Charges de réalisation des automates trop importantes

- Problèmes techniques à contourner
- Ecriture de scripts trop détaillés et difficiles à réutiliser
- Interface applicative instable
- Maintenance des scripts coûteuse

Implication limitée des testeurs

- Testeurs ayant peur du niveau de technicité
- Développeurs / automaticiens ayant élaboré les tests en complète autonomie
- Absence de lien entre le référentiel de test manuel et les tests automatisés

Perception négative du ratio coûts / résultats

- Attentes irréalistes sur le périmètre à couvrir
- Absence de capitalisation
- Résultats de l'automatisation non mesurés

Les facteurs clé d'une automatisation réussie



Equipe intégrée

- L'automatisation consiste en un ensemble d'activités
- La responsabilité de chaque activité est confiée à un acteur de l'équipe (testeur, développeur...)
- Le sponsor (client final ou référent) est impliqué dans la démarche



Pilotage

- Il faut un « porteur » du sujet (coordinateur)
- Un processus de collection et priorisation des demandes est nécessaire : gestion du backlog d'automatisation
- Les coûts et les gains obtenus doivent être mesurés en permanence



Industrialisation

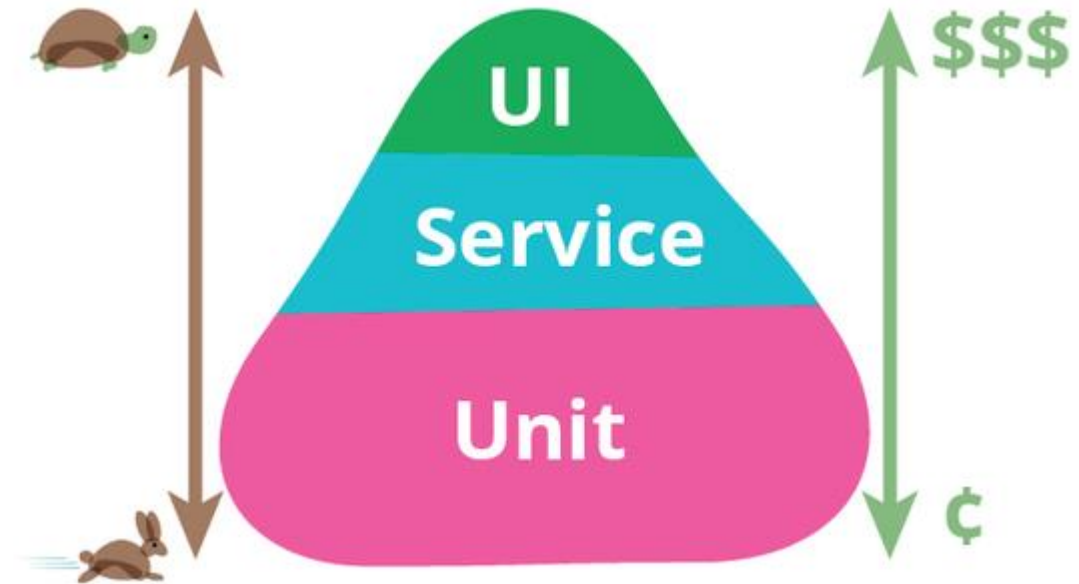
- L'automatisation est gérée comme un projet de développement
- L'objectif poursuivi reste néanmoins la création d'un référentiel de tests réutilisables
- La mise en place d'un référentiel (Framework) est vitale
- L'exécution des tests automatisés doit se faire au plus près du développement

Les niveaux d'automatisation

Quel que soit le cycle de vie utilisé, les tests peuvent être automatisés à plusieurs niveaux :

- Développement : tests unitaires automatisés par les développeurs avant de commit de code
- Intégration : tests de builds automatisés avant de livrer une version aux testeurs
- Système : tests manuels sur l'interface utilisateur, tests de non-régression...
- Acceptation : tests de bout-en-bout, tests de performance...

La principale difficulté reste de savoir quoi automatiser, à quel niveau et dans quelle quantité...



<https://martinfowler.com/bliki/TestPyramid.html>

Les différents types d'outils

Open Source	Editeur
Coût financier très limité	Investissement financier parfois important (licences + support)
Usage souvent unique	Support de plusieurs technologies
Compétences en développement souhaitables	Outils souvent graphiques (facilité d'utilisation)
Intégration limitée ou à fabriquer soi-même	Souvent en suite intégrée avec des outils du même éditeur ou API
Forge Open source existante et support de la communauté	Accès au support et à la maintenance de l'éditeur
Approche projet ou technologie	Approche globale pour toute l'organisation
Expertise interne	Disponibilité des ressources compétentes

Le coût de l'acquisition des licences des solutions commerciales ne représente parfois qu'un faible pourcentage du coût global de l'automatisation !!

1. Installation et configuration Playwright

Installation de l'environnement

<https://playwright.dev/docs/writing-tests>

Pré-requis

1. Node.js: 20.x, 22.x or 24.x.
2. Windows 11+, Windows Server 2019+ or Windows Subsystem for Linux (WSL).
3. macOS 14 (Ventura).
4. Debian 12 / 13, Ubuntu 22.04 / 24.04 (x86-64 or arm64).

Playwright

npm init playwright@latest

Options:

- TypeScript or JavaScript (default: **TypeScript**)
- Tests folder name (default: **tests**, or e2e if tests already exists)
- Add a GitHub Actions workflow (recommended for CI)
- Install Playwright browsers (default: **yes**)

Le fichier [playwright.config](#) configure: les navigateurs, les timeouts, les retries, les projets, et les reporters.

Le fichier package.json contient les dépendances.

VSCode

<https://code.visualstudio.com/>

Installer l'extension Playwright : <https://marketplace.visualstudio.com/items?itemName=ms-playwright.playwright>

<code>playwright.config.ts</code>	<code># Test configuration</code>
<code>package.json</code>	
<code>package-lock.json</code>	<code># Or yarn.lock / pnpm-lock.yaml</code>
<code>tests/</code>	
<code>example.spec.ts</code>	<code># Minimal example test</code>

Installation de l'environnement

```
> npx
> create-playwright

Getting started with writing end-to-end tests with Playwright:
Initializing project in '.'
? Do you want to use TypeScript or JavaScript? ...
> TypeScript
✓ Do you want to use TypeScript or JavaScript? · TypeScript
✓ Where to put your end-to-end tests? · tests
✓ Add a GitHub Actions workflow? (Y/n) · true
✓ Install Playwright browsers (can be done manually via 'npx playwright install')? (Y/n) · true
Initializing NPM project (npm init -y)...
Wrote to C:\Users\h.imhah\OneDrive - ONEPOINT\Bureau\Formation Playwright\TS_Plw\package.json:

{
  "name": "ts_plw",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}

Installing Playwright Test (npm install --save-dev @playwright/test)...
```

Inside that directory, you can run several commands:

```
npx playwright test
Runs the end-to-end tests.
```

```
npx playwright test --ui
Starts the interactive UI mode.
```

```
npx playwright test --project=chromium
Runs the tests only on Desktop Chrome.
```

```
npx playwright test example
Runs the tests in a specific file.
```

```
npx playwright test --debug
Runs the tests in debug mode.
```

```
npx playwright codegen
Auto generate tests with Codegen.
```

We suggest that you begin by typing:

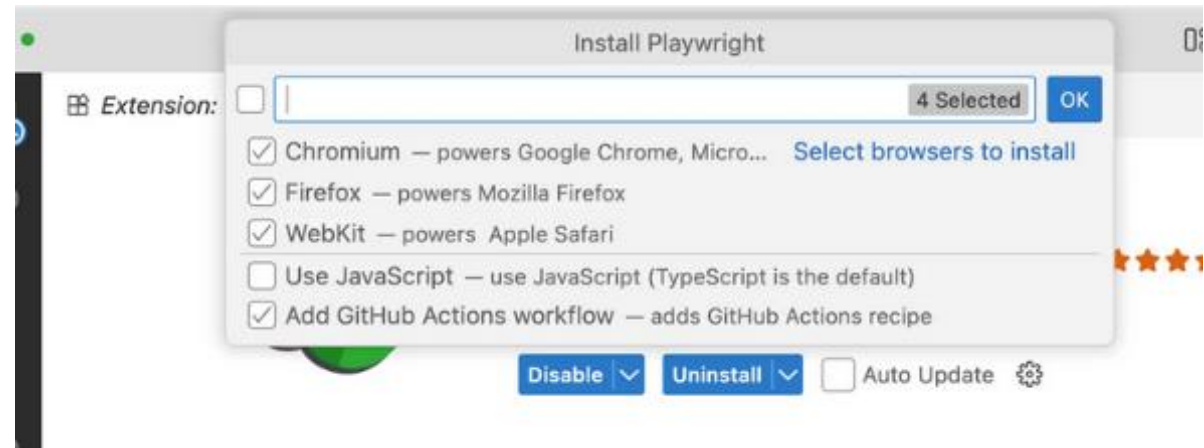
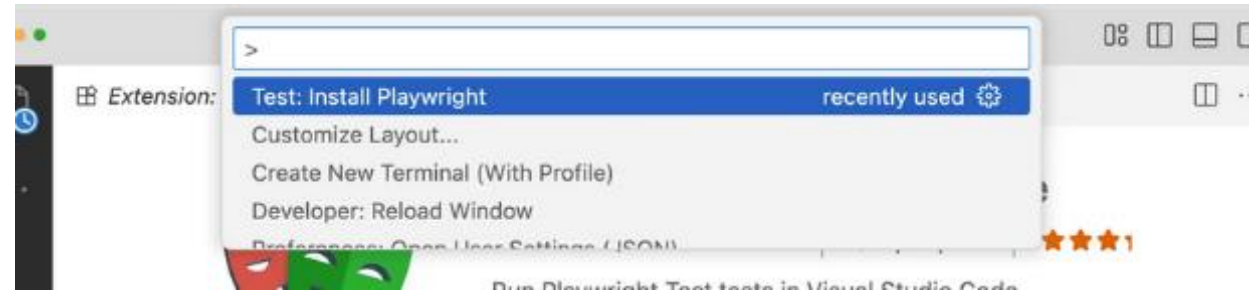
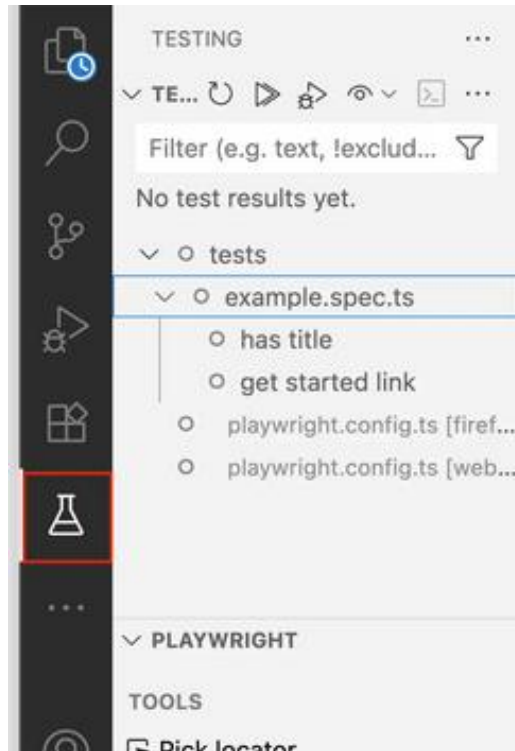
```
npx playwright test
```

And check out the following files:

- `.\tests\example.spec.ts` - Example end-to-end test
- `.\playwright.config.ts` - Playwright Test configuration

Configuration avec l'extension

1. Command Palette (Ctrl+Shift+P or Cmd+Shift+P) et exécuter la commande **Test: Install Playwright**.
2. Configurer le navigateur (la configuration peut être modifier dans playwright.config.ts)
3. Utiliser la barre de test pour exécuter les tests



L'écosystème Playwright

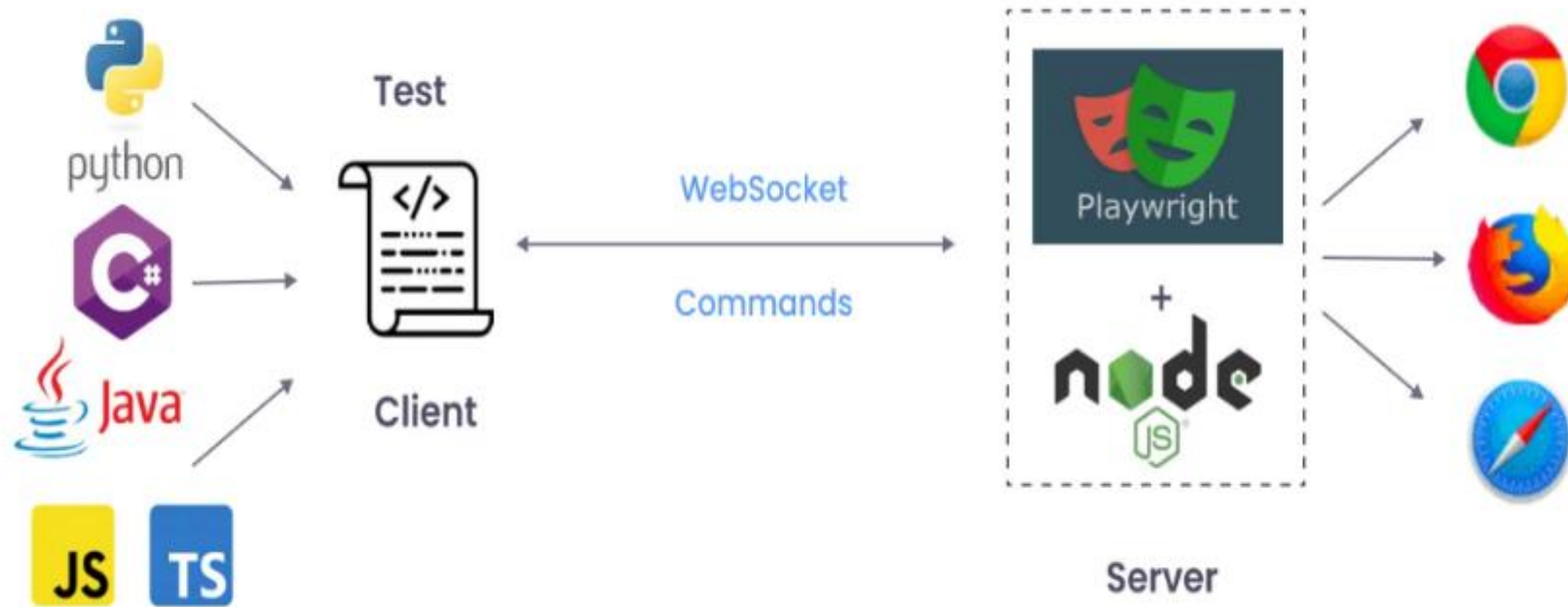
Playwright est un projet open source soutenu par une communauté croissante d'utilisateurs à travers le monde.

Ce projet est maintenu par une équipe active et propose une gamme d'outils pour l'automatisation des tests d'applications web modernes.

La communauté et les contributeurs de Playwright travaillent en continu pour offrir des fonctionnalités avancées, notamment :

- Support multi-navigateurs : Automatisation des tests sur Chrome, Firefox, Safari et Microsoft Edge.
- Test cross-plateforme : Exécution des tests sur différentes plateformes (Windows, MacOS, Linux).
- Prise en charge de plusieurs langages : Disponible en Python, Node.js, Java et C#.
- Tests en parallèle et isolés : Playwright permet de lancer des tests en parallèle dans des contextes de navigateur isolés. Captures d'écran, vidéos et journaux

Architecture Playwright



Présentation de Playwright

Playwright est un Framework d'automatisation des tests de bout en bout à code source ouvert, issu de Puppeteer, qui est basé sur node.js et maintenu par Microsoft.

Dans l'architecture traditionnelle du WebDriver Selenium, chaque requête HTTP est envoyée séparément et reçoit une réponse JSON, ce qui entraîne une communication en va-et-vient qui ralentit l'ensemble du processus.

Playwright, en revanche, utilise une API unique et un seul Web Socket pour communiquer avec tous les pilotes.

C'est pourquoi Playwright se révèle être l'un des Framework d'automatisation les plus rapides et les plus simples dans le domaine des tests logiciels.

Pourquoi utiliser Playwright

- 1. Support de Web Sockets Unique** : Playwright utilise un seul Web Socket pour communiquer avec tous les navigateurs, ce qui améliore la performance et la réactivité des tests. En comparaison, Sélénium peut nécessiter plusieurs canaux de communication, ce qui peut introduire une latence supplémentaire.
- 2. Prise en Charge de Navigateurs Modernes** : Playwright offre une prise en charge native et simultanée de plusieurs navigateurs modernes, notamment Chromium, Firefox et WebKit. Cela garantit une couverture étendue et cohérente des différents environnements de navigation sans nécessiter de configurations complexes. Playwright permet l'émulation d'appareil mobile.
- 3. Tests Plus Stables et Rapides** : Grâce à son architecture moderne, Playwright réduit les problèmes de synchronisation courants dans Selenium, offrant des tests plus stables et plus rapides. Son modèle de gestion des requêtes et des réponses le rend moins sujet aux échecs dus à des délais de chargement ou à des interactions asynchrones.

Pourquoi utiliser Playwright ? Suite

API Moderne et Facile à Utiliser : Playwright propose une API plus moderne et intuitive qui simplifie la rédaction des scripts de test. Ses fonctionnalités intégrées permettent d'interagir facilement avec des éléments de page (attente automatique), de gérer les contextes de navigateur et de manipuler les cookies et le stockage local.

Automatisation de Scénarios Complexes : Playwright permet de simuler des scénarios utilisateur complexes, comme les interactions avec des pop-ups, des notifications et des frames intégrées, de manière plus fluide que Sélénium. Cela est particulièrement utile pour tester des applications web riches en fonctionnalités. Playwright peut intercepter et modifier les requêtes réseau.

Test de Performances et de Sécurité : Avec Playwright, il est plus facile d'intégrer des tests de performance et de sécurité grâce à des outils intégrés et des configurations simplifiées. Il offre également des capacités avancées pour capturer des vidéos et des captures d'écran des tests, facilitant ainsi le diagnostic des problèmes.

2. Identification des objets Web

<https://playwright.dev/docs/locators>



Playwright Test Generator

Test Generator : Playwright inclut un générateur de tests intégré qui permet de créer des scripts d'automatisation rapidement en enregistrant les interactions avec le navigateur. Cette fonctionnalité est conçue pour simplifier la création de tests sans nécessiter une connaissance approfondie de la programmation. Il possède lui-même sa propre interface de développement

Richesse des fonctionnalités : Playwright permet également d'ajuster et d'améliorer les scripts générés pour des scénarios complexes, en tirant parti de ses capacités avancées comme la gestion des contextes de navigation et des interactions dynamiques.

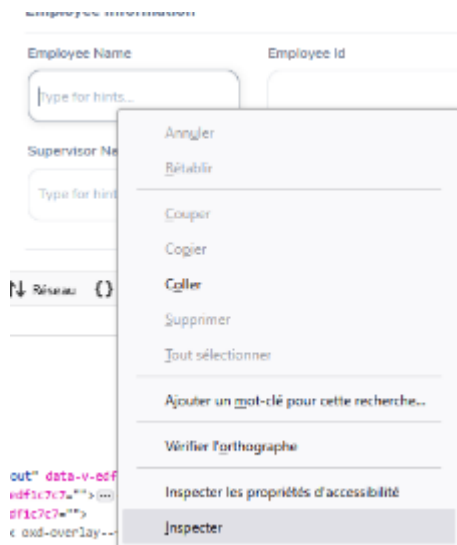
Inconvénient :

Les localisateurs générés ne sont pas toujours pertinents.

Retravailler les localisateurs avec les devtools du navigateur ou avec un plugin.

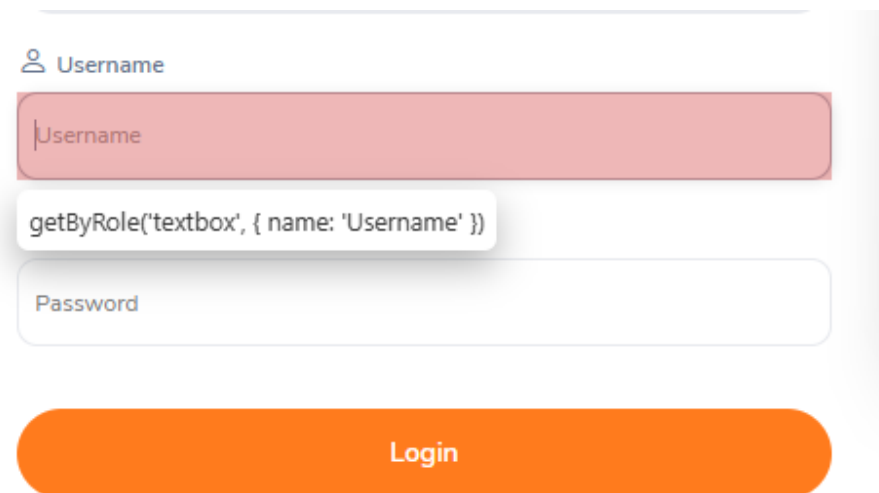
Exécuter :

npx playwright codegen



21

```
Playwright Inspector - https://opensource-demo.orangehrmlive.com/web/index.php/auth/login
Record | [Icons] | Target: Test Runner
1 import { test, expect } from '@playwright/test';
2
3 test('test', async ({ page }) => {
4   await page.goto('https://opensource-demo.orangehrmlive.com/web/index.php/auth/login');
5   await page.getByRole('textbox', { name: 'Username' }).click();
6   await page.getByRole('textbox', { name: 'Username' }).fill('admin');
7   await page.getByRole('textbox', { name: 'Password' }).click();
8   await page.getByRole('textbox', { name: 'Password' }).fill('admin123');
9   await page.getByRole('button', { name: 'Login' }).click();
10 });
```



Localisateurs courants

	Selenium	Playwright	Explication
ID	<code>driver.find_element(By.ID, "id_value")</code>	<code>page.locator("#id_value")</code>	Identifie un élément par son attribut ID unique.
Nom	<code>driver.find_element(By.NAME, "name_value")</code>	<code>page.locator("[name='name_value']")</code>	Cible un élément par son attribut name.
Classe	<code>driver.find_element(By.CLASS_NAME, "class_value")</code>	<code>page.locator(".class_value")</code>	Sélectionne des éléments par leur classe CSS.
Texte du lien	<code>driver.find_element(By.LINK_TEXT, "link_text")</code>	<code>page.get_by_text("link_text")</code>	Trouve des liens par leur texte visible.
XPath	<code>driver.find_element(By.XPATH, "xpath_expression")</code>	<code>page.locator("xpath=xpath_expression")</code>	Utilise des expressions XPath pour localiser des éléments.
CSS Selector	<code>driver.find_element(By.CSS_SELECTOR, "css_selector")</code>	<code>page.locator("css_selector")</code>	Emploie des sélecteurs CSS pour cibler des éléments.
Label	Non disponible nativement	<code>page.get_by_label("label_text")</code>	Trouve des éléments par le texte de leur label associé.
Placeholder	Non disponible nativement	<code>page.get_by_placeholder("placeholder_text")</code>	

Localisateurs spécifiques Playwright

Syntaxe Python
A modifier

page.get_by_text() : Localise un élément par son texte visible.

page.get_by_role() : Trouve des éléments basés sur leur rôle ARIA.

page.get_by_label() : Cible des éléments associés à un label spécifique.

page.get_by_placeholder() : Localise des éléments input par leur texte placeholder.

page.get_by_alt_text() : Trouve des images par leur texte alternatif.

page.get_by_title() : Sélectionne des éléments par leur attribut title.

page.get_by_test_id() : Cible des éléments avec un attribut data-testid spécifique.

page.frame_locator() : Localise des éléments à l'intérieur d'un iframe.

page.get_by_aria_label() : Trouve des éléments par leur attribut aria-label.

page.get_by_aria_role() : Sélectionne des éléments basés sur leur rôle ARIA, similaire à get_by_role mais plus spécifique.

Localisateurs CSS

Sélecteur	Description
div	Recherche tous les éléments de type div
div.class	Recherche tous les éléments de type div contenant une classe appelée "class"
.class	Recherche tous les éléments contenant une classe appelée "class"
#objectId	Recherche tous les éléments qui ont un Id égal à "objectId"
input[name]	Recherche tous les éléments de type "input" contenant l'attribut "name"
input[name="a"]	Recherche tous les éléments de type "input" dont l'attribut "name" prend la valeur "a"
input[name*="a"]	Recherche tous les éléments de type "input" dont l'attribut "name" contient la valeur "a"

Sélecteur	Description
p:nth-child(n)	Recherche tous les éléments de type "p" qui sont les n-ièmes enfants de leur parent
p:nth-of-type(n)	Recherche tous les éléments de type "p" qui sont les n-ièmes enfants de type "p" de leur parent

https://www.w3schools.com/cssref/css_selectors.php

Localisateurs XPATH

Expression	Description
/html/body	Recherche la section body directement sous la racine html
//input	Recherche tous les éléments input dans la page
//input[@type='text']	Recherche tous les éléments input dont l'attribut type prend la valeur 'text'
//input[contains(@class, 'result')]	Recherche tous les éléments input dont l'attribut 'class' contient la valeur 'result'
//input[starts-with(@class, 'result')]	Recherche tous les éléments input dont l'attribut 'class' commence par la valeur 'result'
//input[not(contains(@class, 'hidden'))]	Recherche tous les éléments input dont l'attribut 'class' n'a pas la valeur 'hidden'
//img[@width<20][@height<20] //img[@width<20 and @height<20]	Condition AND
//input[@name='q' or @id='input']	Condition OR
//*[text()='Search']	Recherche tous les éléments dont le texte est égal à 'Search'
//*[contains(text(),'Search')]	Recherche tous les éléments dont le texte contient 'Search'
//div[contains(.,'Search')]	Recherche tous les éléments de type div et leurs descendants dont le texte contient 'Search'

https://www.w3schools.com/xml/xpath_syntax.asp

Commandes utiles

Commande	Action	Type de retour
<code>await page.goto("url")</code>	Dirige le navigateur vers une adresse spécifique.	<code>Promise<Response null></code>
<code>await page.url()</code>	Récupère l'URL actuelle de la page.	<code>string</code>
<code>await page.title()</code>	Récupère le titre de l'onglet (balise <code><title></code>).	<code>Promise<string></code>
<code>await page.reload()</code>	Actualise la page actuelle.	<code>Promise<Response null></code>
<code>await page.screenshot({path: '...'})</code>	Prend une capture d'écran et l'enregistre sur le disque.	<code>Promise<Buffer></code>
<code>await page.setViewportSize({w, h})</code>	Change la taille de la fenêtre (ex: simuler un mobile).	<code>Promise<void></code>
<code>await browser.close()</code>	Ferme le navigateur et termine la session.	<code>Promise<void></code>

Commande	Action	Retour (Type)
<code>await locator.clear()</code>	Efface le contenu d'un champ de saisie (input ou textarea).	<code>Promise<void></code>
<code>await locator.click()</code>	Simule un clic sur l'élément.	<code>Promise<void></code>
<code>await page.locator('...').all()</code>	Récupère tous les éléments correspondant au sélecteur sous forme d'une liste.	<code>Promise<Locator[]></code>
<code>await locator.fill('texte')</code>	Remplit un champ avec le texte spécifié (plus rapide que <code>type</code>).	<code>Promise<void></code>
<code>await locator.boundingBox()</code>	Récupère les coordonnées (x, y) et la taille (width, height) de l'élément.	<code>Promise<Box null></code>
<code>await locator.getAttribute('nom')</code>	Récupère la valeur d'un attribut HTML (ex: <code>src</code> , <code>href</code>).	<code>Promise<string null></code>
<code>await locator.textContent()</code>	Récupère tout le texte contenu à l'intérieur de l'élément.	<code>Promise<string null></code>



Exercice 1 : Utilisation codegen et localisateurs

- 0.1. Lancer npx playwright codegen
 - 0.2. Réaliser le test manuel sur orangehrm (<https://opensource-demo.orangehrmlive.com>)
Login → Création Employé → Saisie du détail → Recherche Employé par nom → Modification des coordonnées →
Recherche Employé par matricule → Suppression de l'employé → Logout
 - 0.4. Analyser le code généré et modifier les localisateurs non pertinents.
 - 0.5. Intégrer Login et Logout dans un test et l'exécuter.
 - 0.6. Analyser les résultats et corriger les erreurs
 - 0.7. Exécuter le test et enregistrer la vidéo (<https://playwright.dev/docs/videos>)
- Utiliser le code de l'exemple <https://playwright.dev/docs/intro> comme modèle

```
fullyParallel: false, // Files run sequentially
workers: 1,
timeout: 50 * 1000, // 60s per test (default: 30s) [web:50]
expect: { timeout: 10 * 1000 }, // 10s per assertion (default: 5s)
launchOptions: {
  slowMo: 1000, // 1000ms delay between actions
},
```

Configuration recommandée dans playwright.config.ts



3. Gestion de session

<https://playwright.dev/docs/auth>



Isolation totale

Playwright applique le principe de l'isolation totale des tests pour permettre le parallélisme, pour plus de fiabilité et de déterminisme dans les tests.

Fiabilité	Si le Test A échoue ou modifie un réglage, cela n'affecte pas le Test B. On évite les "effets de bord".
Parallélisme	Puisque les tests sont indépendants, Playwright peut ouvrir 5 navigateurs en même temps et lancer 5 tests simultanément sans qu'ils ne se mélangent les pinceaux.
Déterminisme	Tu es sûr que ton test part toujours du même point de départ. Si ton test échoue, c'est à cause du code, pas à cause d'un vieux cookie qui traînait

Il est donc nécessaire de mettre en place un mécanisme de gestion de la session pour éviter la connexion avant chaque test.

Cela permet d'enregistrer "l'état" du navigateur (les **cookies** et le **local storage**) dans un fichier JSON. Pour tous les tests suivants, Playwright charge ce fichier pour simuler l'état connecté.



Gestion du sessionStorage

1. Crée un dossier .auth
2. Crée un fichier nommé auth.setup.ts
3. Configurer playwright.config.ts

4. Pour tester le login / logout ajouter

```
// Reset storage state for this file to avoid being authenticated
test.use({ storageState: { cookies: [], origins: [] } });
```

```
import { test as setup } from '@playwright/test';
const authFile = '.auth/user.json';

setup('authenticate', async ({ page }) => {
  // 1. Connexion à l'application

  // 2. Sauvegarder l'état de connexion (cookies, etc.)
  await page.context().storageState({ path: authFile });
});
```

```
projects: [
  { name: 'setup',
    testMatch: /\.*\bsetup\.ts/ },

  { name: 'chromium',
    use: { ...devices['Desktop Chrome'],
      storageState: '.auth/user.json',
    },
    dependencies: ['setup'], // lance le setup avant
  },
],
```

Exercice 2 : sessionStorage

- 0.1. Reprendre le code généré dans l'exercice 1
- 0.2. Découpé en plusieurs tests et exécuter les tests. Les tests échouent (sauf le 1^{er})
- 0.3. Ajouter la gestion de session
- 0.4. Exécuter à nouveau les tests et vérifier que les tests sont en succès.
- 0.5. Tester le login / logout avec les cas d'erreur



4. Les assertions

<https://playwright.dev/docs/test-assertions>



Les assertions dans Playwright

Les assertions Playwright permettent de vérifier si des conditions spécifiques sont remplies dans l'application testée. Elles sont essentielles pour confirmer que l'application se comporte comme prévu lors des tests.

Types d'assertions

Assertions de base

```
# Vérifier si une valeur est vraie
expect(success).toBeTruthy();

# Vérifier l'égalité
expect(result).toEqual(val);

# Vérifier si une chaîne contient une
sous-chaîne
expect(text).toContain('...');
```

Assertions spécifiques au web

```
# Vérifier le titre de la page
await expect(page).toHaveTitle('...');

# Vérifier la visibilité d'un élément
await expect(page.locator('button#submit')).toBeVisible();

# Vérifier le texte d'un élément
expect(page.locator("h1")).to_have_text("Welcome")

# Vérifier l'URL de la page
await expect(page).toHaveURL('https://example.com/dashboard');
```

Assertions sur les localisateurs

```
# Vérifier si une case à cocher est cochée
await expect(page.locator('#terms')).toBeChecked();

# Vérifier si un élément est activé
await expect(page.locator('button#submit')).toBeEnabled();

# Vérifier le nombre d'éléments
await expect(page.locator('li.item')).toHaveCount(5);

# Vérifier la valeur d'un champ de saisie
await
expect(page.locator('input#email')).toHaveValue('user@example.com');
```



Les assertions courantes

- **toBe()** : Vérifie l'égalité stricte pour les valeurs primitives
- **toEqual()** : Effectue une comparaison profonde pour les objets et les tableaux
- **toContain()** : Vérifie si une chaîne ou un tableau contient une valeur spécifique
- **toBeTruthy()** : Vérifie si une valeur est considérée comme vraie
- **toHaveText()** : Vérifie le texte d'un élément web
- **toHaveTitle()** : Vérifie le titre de la page
- **toBeVisible()** : Vérifie si un élément est visible sur la page
- **toHaveLength()** : Vérifie le nombre d'éléments correspondant à un sélecteur
- **toHaveAttribute()** : Vérifie si un élément possède un attribut spécifique
- **toHaveValue()** : Vérifie la valeur d'un champ de formulaire



Exercice 2 : Ajouter les assertions

- 0.1. Reprendre le code généré dans l'exercice 1
- 0.2. Compléter les tests avec : Création Employé → Saisie du détail → Recherche Employé par nom
- 0.4. Ajouter les assertions pertinentes
- 0.5. Rendre le code modulaire en créant une librairie pour OrangeHRM
- 0.6. Exécuter le test, analyser les résultats et corriger les erreurs

5. Utilisations des données



Utilisation de données (CSV/JSON)

En TypeScript le **Data Driven Testing** se fait via une boucle simple ou l'import direct de fichiers JSON

Nécessite de configurer dans tsconfig.json

```
"resolveJsonModule": true,
```

```
import data from './data.json';
```

```
for (const user of data) {  
    test(`Test pour ${user.name}`, async ({ page }) => {  
        await page.fill('#user', user.username);  
    });  
}
```



Utilisation de faker

<https://github.com/faker-js/faker>

La librairie **@faker-js/faker** est une librairie qui permet de générer des données réalistes et aléatoires (noms, adresses, emails, dates) afin de peupler des formulaires ou des bases de données de test.

Elle permet d'éviter d'utiliser des données réelles ou répétitives (comme "test1", "test2") en offrant des fonctions prêtes à l'emploi couvrant divers domaines.



Features

- 🧑 Person - Generate Names, Genders, Bios, Job titles, and more.
- 📍 Location - Generate Addresses, Zip Codes, Street Names, States, and Countries!
- 📅 Date - Past, present, future, recent, soon... whenever!
- 💰 Finance - Create stubbed out Account Details, Transactions, and Crypto Addresses.
- 🛒 Commerce - Generate Prices, Product Names, Adjectives, and Descriptions.
- 🦾 Hacker - "Try to reboot the SQL bus, maybe it will bypass the virtual application!"
- 📄 Number and String - Of course, we can also generate random numbers and strings.
- 🌍 Localization - Pick from over 70 locales to generate realistic looking Names, Addresses, and Phone Numbers.

```
import { faker } from '@faker-js/faker';

function formatDateToYYYYDDMM(dateStr: string): string {
  const [year, month, day] = dateStr.split('-');
  return `${year}-${day}-${month}`;
}

const employee = {
  firstName: faker.person.firstName(),
  lastName: faker.person.lastName(),
  employeeId: faker.string.numeric(6),
  dateOfBirth: faker.date.birthdate({ mode: 'age', min: 18, max: 65 }).toISOString().split('T')[0],
  empld: ""
};
```



Exercice 3 : data driven test

- 0.1. Reprendre le code généré dans l'exercice 2
- 0.2. Compléter les tests avec : Création Employé → Saisie du détail → Recherche Employé par nom
- 0.3. Créer 20 employés à partir d'un fichier json (utiliser Faker pour générer les noms prénoms automatiquement)
- 0.4. Exécuter le test, analyser les résultats et corriger les erreurs



6. Page Object Model

<https://playwright.dev/docs/pom>



Modularité des tests

Le POM consiste à créer une classe TypeScript pour chaque page de l'application.

Cette classe contient :

- Les sélecteurs (les éléments de la page).
- Les méthodes d'action (ce qu'on peut faire sur cette page, ex: login(), logout()).

Avantage	Description
Maintenance facile	Si un bouton change de place, le sélecteur est modifié une seule fois dans la classe de la page.
Lisibilité	Tests plus lisibles : <code>await loginPage.login('admin', '123')</code> au lieu de 5 lignes de <code>page.locator(...)</code> .
Réutilisabilité	N'importe quel test peut appeler les méthodes d'une page déjà créée.



Exercice 5 : Page object Model

0.1. Créer les pages POM : `pages/login.page.ts` et `pages/dashboard.page.ts`

0.2. Réécrire le test `login/logout` en utilisant ces pages

0.3. Utiliser CoPilot pour réécrire le test `employee.spec.ts` avec les pages

- `pages/login.page.ts`

- `pages/dashboard.page.ts`

- `pages/add-employee.page.ts`

- `pages/employee-details.page.ts`

- `pages/employee-list.page.ts`

- `pages/employee-contact.page.ts`



7. Playwright Test Agents

<https://playwright.dev/docs/test-agents>



Agents MCP Playwright

Playwright intègre nativement trois agents spécialisés pour vos tests :

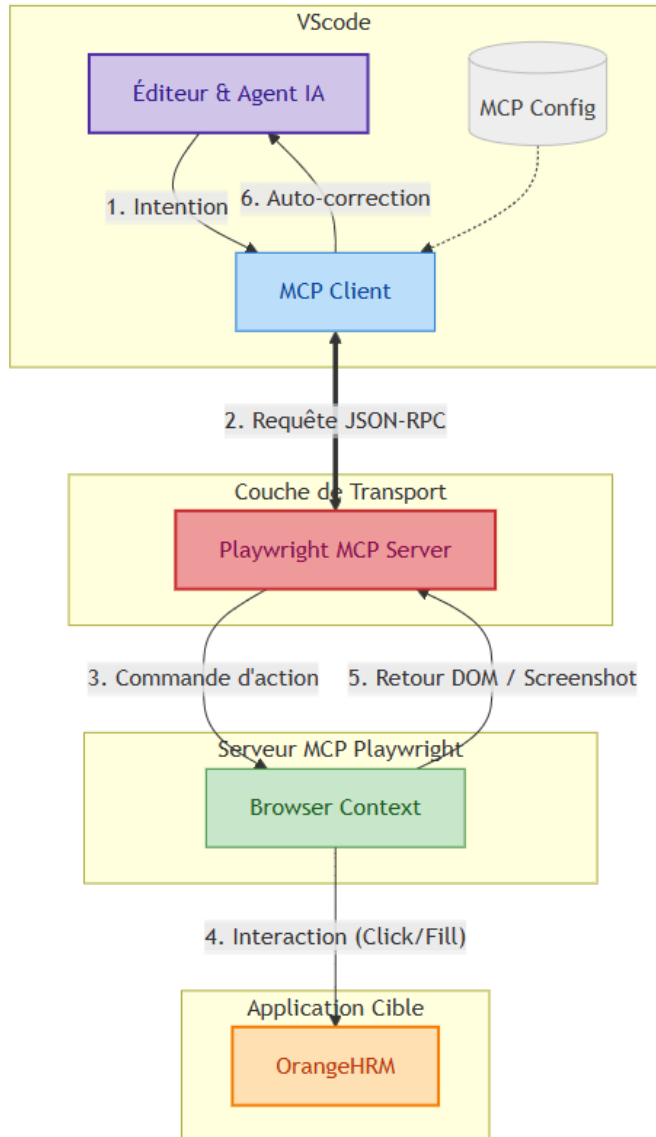
 **planner** : explore l'application et rédige un plan de test au format Markdown.

 **generator** : transforme ce plan Markdown en fichiers de test Playwright exploitables.

 **healer** : exécute la suite de tests et répare automatiquement ceux qui échouent.

Ces agents peuvent fonctionner seuls ou en boucle pour garantir une couverture de test complète et auto-gérée.

Fonctionnement du MCP Playwright



- **Utiliser le "Sitemap" en Ressource locale**

Utiliser une **ressource statique** (fichier JSON ou Markdown) décrivant les sélecteurs principaux.

L'IA n'aura plus besoin d'appeler l'outil d'analyse du DOM pour trouver le menu PIM ; elle aura l'information directement dans son contexte de départ.

- **Privilégier l'Accessibility Tree au Screenshot**

playwright_screenshot envoie une image haute résolution au LLM, ce qui coûte très cher en tokens "vision". L'outil d'analyse du DOM/Accessibility Tree renvoie du texte.

Préciser dans le prompt : *"N'utilise la capture d'écran que si tu es bloqué après deux tentatives d'analyse du DOM."*

- **Restreindre la portée de l'inspection**

Par défaut, Playwright peut renvoyer l'intégralité du DOM.

Demandez à l'IA d'utiliser des sélecteurs ciblés dès le départ. Au lieu de *"Analyse la page"*, dites : *"Examine uniquement la zone du formulaire de création d'employé pour trouver le bouton de validation."*

- **Enchaîner les instructions (Batching)**

Chaque interaction MCP (aller-retour entre Antigravity et le serveur) renvoie une partie du contexte de la page.

Donnez des missions complètes. *"Connecte-toi avec Admin/admin123, va dans le module PIM et ouvre le formulaire d'ajout d'employé. Ne me redonne la main qu'une fois le formulaire affiché."*

Initialisation des agents

`npx playwright init-agents --loop=vscode`

`npx playwright init-agents --loop=vscode`

📁 Using project "chromium" as a primary project

📄 specs\README.md - directory for test plans

🌱 tests\seed.spec.ts - default environment seed file

🤖 .github\agents\playwright-test-generator.agent.md - agent definition

🤖 .github\agents\playwright-test-healer.agent.md - agent definition

🤖 .github\agents\playwright-test-planner.agent.md - agent definition

🔧 .vscode\mcp.json - mcp configuration

🔧 .github\workflows\copilot-setup-steps.yml - GitHub Copilot setup steps

🔧 TODO: GitHub > Settings > Copilot > Coding agent > MCP configuration

```
{
  "mcpServers": {
    "playwright-test": {
      "type": "stdio",
      "command": "npx",
      "args": [
        "playwright",
        "run-test-mcp-server"
      ],
      "tools": [
        "*"
      ]
    }
  }
}
```

✅ Done.

Agent Planner

Conseil : utiliser un fichier Site-Map (json) pour limiter l'exploration de l'agent

Connecte-toi sur <https://opensource-demo.orangehrmlive.com> avec le login admin et le mot de passe admin123

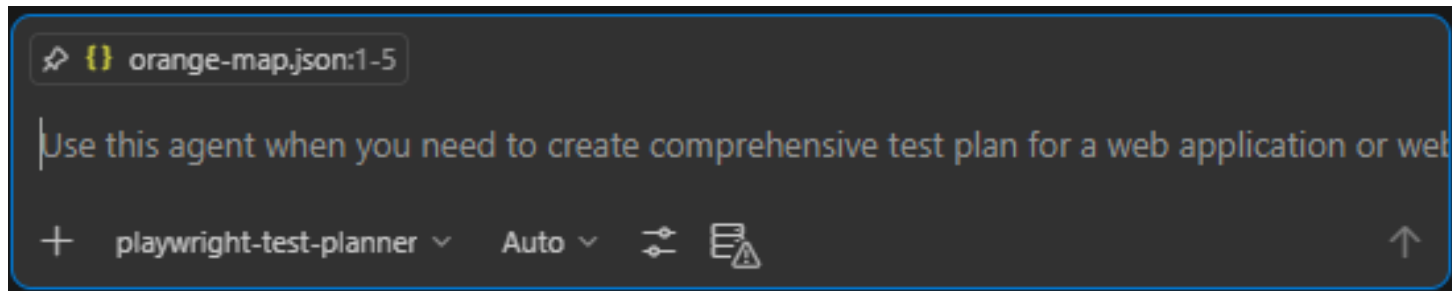
En utilisant @orange-map.json, explore le module de création d'employés et rédige une procédure de test manuel détaillée avec : Étapes, Données d'entrée et Résultats attendus.

Limite-toi aux tests suivants :

- création d'un nouvel employé,
- saisie des Personal Details
- saisie des Contact Details

orange-map.json

```
{
  "modules": {
    "PIM": {
      "role": "link",
      "name": "PIM",
      "submenus": [
        "Add Employee",
        "Employee List",
        "Personal Details",
        "Contact Details"
      ]
    }
  }
}
```



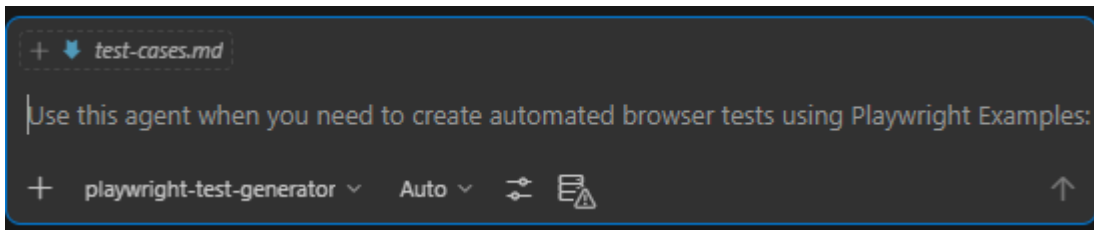
Agent Generator

Conseil : Utiliser la documentation des tests manuels générés par Planner et imposer le modèle Page Object

Génère un fichier de spec « employee.spec.ts » avec 3 tests :

1. Add Employee and fill Personal Details,
2. Search Employee and fill Contact Details,
3. Search Employee and Delete Employee

Utilise le framework Playwright Test et le modèle Page Object.



Agent Healer

Lorsqu'un test échoue, l'agent **Healer** prend le relais pour le réparer :

1. **Analyse** : Il rejoue les étapes en échec et inspecte l'interface (UI) pour retrouver les éléments ou les flux équivalents.
2. **Correction** : Il propose un correctif (mise à jour du sélecteur, ajustement des temps d'attente ou correction des données).
3. **Validation** : Il relance le test en boucle jusqu'à ce qu'il réussisse (ou que les limites de sécurité soient atteintes).

Entrée requise : Le nom du test en échec.



8. Cas pratique

Cas pratique : taiga.io

Reprendre le projet créé dans le TD Postman sur le site taiga.io (<https://tree.taiga.io>)

Automatiser le scénario suivant :

1. Se connecter à l'application
2. Sélectionner le projet
3. Afficher la liste des anomalies (issues) du projet
4. Créer une anomalie : Subject / Description / Type / Severity / Priority
5. Créer une liste 20 d'anomalies avec une variation sur le type, la sévérité et la priorité, à partir d'un fichier JSON
6. Appliquer le filtre Type = Bug et Severity = Critical
7. Rechercher une anomalie par la référence et la supprimer



Merci

